

如果人物的健康可纯粹根据该人物 public 接口得来的信息加以计算, 这没有问题, 但如果需要 non-public 信息进行精确计算, 就有问题了。实际上任何时候当你将 class 内的某个机能(也许取自某个成员函数)替换为 class 外部的某个等价机能(也许取自某个 non-member non-friend 函数或另一个 class 的 non-friend 成员函数), 这都是潜在争议点。这个争议将持续至本条款其余篇幅, 因为我们即将考虑的所有替代设计也都涉及使用 GameCharacter 继承体系外的函数。

一般而言, 唯一能够解决“需要以 non-member 函数访问 class 的 non-public 成分”的办法就是: 弱化 class 的封装。例如 class 可声明那个 non-member 函数为 friends, 或是为其实现的某一部分提供 public 访问函数(其他部分则宁可隐藏起来)。运用函数指针替换 virtual 函数, 其优点(像是“每个对象可各自拥有自己的健康计算函数”和“可在运行期改变计算函数”)是否足以弥补缺点(例如可能必须降低 GameCharacter 封装性), 是你必须根据每个设计情况的不同而抉择的。

藉由 tr1::function 完成 **Strategy** 模式

一旦习惯了 templates 以及它们对隐式接口(见条款 41)的使用, 基于函数指针的做法看起来便过分苛刻而死板了。为什么要求“健康指数之计算”必须是个函数, 而不能是某种“像函数的东西”(例如函数对象)呢? 如果一定得是函数, 为什么不能够是个成员函数? 为什么一定得返回 int 而不是任何可被转换为 int 的类型呢?

如果我们不再使用函数指针(如前例的 healthFunc), 而是改用一个类型为 tr1::function 的对象, 这些约束就全都挥发不见了。就像条款 54 所说, 这样的对象可持有(保存)任何可调用物(callable entity, 也就是函数指针、函数对象、或成员函数指针), 只要其签名式兼容于需求端。以下将刚才的设计改为使用 tr1::function:

```
class GameCharacter; //如前
int defaultHealthCalc(const GameCharacter& gc); //如前
class GameCharacter {
public:
    //HealthCalcFunc 可以是任何“可调用物”(callable entity), 可被调用并接受
    //任何兼容于 GameCharacter 之物, 返回任何兼容于 int 的东西。详下。
    typedef std::tr1::function<int (const GameCharacter&)> HealthCalcFunc;
```

```

explicit GameCharacter(HealthCalcFunc hcf = defaultHealthCalc)
    : healthFunc(hcf)
{
    int healthValue() const
    { return healthFunc( * this); }
    ...
private:
    HealthCalcFunc healthFunc;
};

```

如你所见, HealthCalcFunc 是个 **typedef**, 用来表现 `trl::function` 的某个具现体, 意味该具现体的行为像一般的函数指针。现在我们靠近一点瞧瞧 HealthCalcFunc 是个什么样的 **typedef**:

```
std::trl::function<int (const GameCharacter&)>
```

这里我把 `trl::function` 具现体 (*instantiation*) 的目标签名式 (*target signature*) 以不同颜色强调出来。那个签名代表的函数是“接受一个 **reference** 指向 `const GameCharacter`, 并返回 `int`”。这个 `trl::function` 类型 (也就是我们所定义的 HealthCalcFunc 类型) 产生的对象可以持有 (保存) 任何与此签名式兼容的可调用物 (**callable entity**)。所谓兼容, 意思是这个可调用物的参数可被隐式转换为 `const GameCharacter&`, 而其返回类型可被隐式转换为 `int`。

和前一个设计 (其 `GameCharacter` 持有的是函数指针) 比较, 这个设计几乎相同。唯一不同的是如今 `GameCharacter` 持有一个 `trl::function` 对象, 相当于一个指向函数的泛化指针。这个改变如此细小, 我总说它没有什么外显影响, 除非客户在“指定健康计算函数”这件事上需要更惊人的弹性:

```

short calcHealth(const GameCharacter&);    //健康计算函数;
                                         //注意其返回类型为 non-int

struct HealthCalculator {                //为计算健康而设计的函数对象
    int operator() (const GameCharacter&) const
    { ... }
};

class GameLevel {
public:
    float health(const GameCharacter&) const;    //成员函数, 用以计算健康;
    ...                                         //注意其 non-int 返回类型
};

class EvilBadGuy: public GameCharacter {    //同前
    ...
};

```

```

class EyeCandyCharacter: public GameCharacter {    //另一个人物类型;
    ...                                           //假设其构造函数与
};                                               //EvilBadGuy 同

EvilBadGuy ebg1(calcHealth);                    //人物 1, 使用某个
                                                // 函数计算健康指数

EyeCandyCharacter eccl(HealthCalculator());      //人物 2, 使用某个
                                                // 函数对象计算健康指数

GameLevel currentLevel;
...
EvilBadGuy ebg2(                                //人物 3, 使用某个
    std::tr1::bind(&GameLevel::health,          // 成员函数计算健康指数
        currentLevel,
        _1)                                     //详见以下
);

```

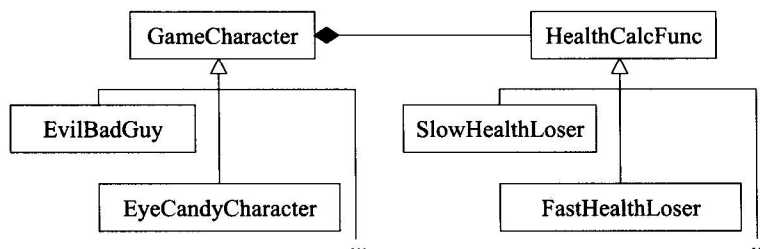
就我个人而言, 当我发现 `tr1::function` 允许我们做的事时, 非常吃惊。它让我浑身震颤。如果你没有这样的感觉, 也许是你早已曾经惊叹 `tr1::bind` 所发生的事情。请允许我稍加解释。

首先我要表明, 为计算 `ebg2` 的健康指数, 应该使用 `GameLevel` class 的成员函数 `health`。好, `GameLevel::health` 宣称它自己接受一个参数 (那是个 `reference` 指向 `GameCharacter`), 但它实际上接受两个参数, 因为它也获得一个隐式参数 `GameLevel`, 也就是 `this` 所指的那个。然而 `GameCharacters` 的健康计算函数只接受单一参数: `GameCharacter` (这个对象将被计算出健康指数)。如果我们使用 `GameLevel::health` 作为 `ebg2` 的健康计算函数, 我们必须以某种方式转换它, 使它不再接受两个参数 (一个 `GameCharacter` 和一个 `GameLevel`), 转而接受单一参数 (一个 `GameCharacter`)。在这个例子中我们必然会想要使用 `currentLevel` 作为“`ebg2` 的健康计算函数所需的那个 `GameLevel` 对象”, 于是我们将 `currentLevel` 绑定为 `GameLevel` 对象, 让它在“每次 `GameLevel::health` 被调用以计算 `ebg2` 的健康”时被使用。那正是 `tr1::bind` 的作为: 它指出 `ebg2` 的健康计算函数应该总是以 `currentLevel` 作为 `GameLevel` 对象。

我跳过了一大堆细节, 像是为什么 “_1” 意味“当为 `ebg2` 调用 `GameLevel::health` 时系以 `currentLevel` 作为 `GameLevel` 对象”。这样的细节不难阐述, 但它们会分散我要说的根本重点: 若以 `tr1::function` 替换函数指针, 吾人将因此允许客户在计算人物健康指数时使用任何兼容的可调用物 (*callable entity*)。如果这还不酷, 什么是酷?

古典的 **Strategy** 模式

如果你对设计模式 (design patterns) 比对 C++ 的酷劲更有兴趣, 我告诉你, 传统(典型)的 **Strategy** 做法会将健康计算函数做成一个分离的继承体系中的 virtual 成员函数。设计结果看起来像这样:



如果你并未精通 UML 符号, 别担心, 这图只是告诉你 GameCharacter 是某个继承体系的根类, 体系中的 EvilBadGuy 和 EyeCandyCharacter 都是 **derived classes**; HealthCalcFunc 是另一个继承体系的根类, 体系中的 SlowHealthLoser 和 FastHealthLoser 都是 **derived classes**, 每一个 GameCharacter 对象都内含一个指针, 指向一个来自 HealthCalcFunc 继承体系的对象。

下面是对应的代码骨干:

```

class GameCharacter;           //前置声明 (forward declaration)
class HealthCalcFunc {
public:
    ...
    virtual int calc(const GameCharacter& gc) const
    { ... }
    ...
};

HealthCalcFunc defaultHealthCalc;

class GameCharacter {
public:
    explicit GameCharacter(HealthCalcFunc* phcf = &defaultHealthCalc)
        : pHealthCalc(phcf)
    {}
    int healthValue() const
    { return pHealthCalc->calc(*this); }
    ...
private:
    HealthCalcFunc* pHealthCalc;
};
  
```

这个解法的吸引力在于,熟悉标准 **Strategy** 模式的人很容易辨认它,而且它还提供“将一个既有的健康算法纳入使用”的可能性——只要为 `HealthCalcFunc` 继承体系添加一个 `derived class` 即可。

摘要

本条款的根本忠告是,当你为解决问题而寻找某个设计方法时,不妨考虑 `virtual` 函数的替代方案。下面快速重点复习我们验证过的几个替代方案:

- 使用 `non-virtual interface (NVI)` 手法,那是 **Template Method** 设计模式的一种特殊形式。它以 `public non-virtual` 成员函数包裹较低访问性(`private` 或 `protected`)的 `virtual` 函数。
- 将 `virtual` 函数替换为“函数指针成员变量”,这是 **Strategy** 设计模式的一种分解表现形式。
- 以 `tr1::function` 成员变量替换 `virtual` 函数,因而允许使用任何可调用物(`callable entity`)搭配一个兼容于需求的签名式。这也是 **Strategy** 设计模式的某种形式。
- 将继承体系内的 `virtual` 函数替换为另一个继承体系内的 `virtual` 函数。这是 **Strategy** 设计模式的传统实现手法。

以上并未彻底而详尽地列出 `virtual` 函数的所有替换方案,但应该足够让你知道的确有不少替换方案。此外,它们各有其相对的优点和缺点,你应该把它们全部列入考虑。

为避免陷入面向对象设计路上因常规而形成的凹洞中,偶而我们需要对着车轮猛推一把。这个世界还有其他许多道路,值得我们花时间加以研究。

请记住

- `virtual` 函数的替代方案包括 `NVI` 手法及 **Strategy** 设计模式的多种形式。`NVI` 手法自身是一个特殊形式的 **Template Method** 设计模式。
- 将机能从成员函数移到 `class` 外部函数,带来的一个缺点是,非成员函数无法访问 `class` 的 `non-public` 成员。
- `tr1::function` 对象的行为就像一般函数指针。这样的对象可接纳“与给定之目标签名式(`target signature`)兼容”的所有可调用物(`callable entities`)。

条款 36：绝不重新定义继承而来的 non-virtual 函数

Never redefine an inherited non-virtual function.

假设我告诉你，class D 系由 class B 以 public 形式派生而来，class B 定义有一个 public 成员函数 mf。由于 mf 的参数和返回值都不重要，所以我假设两者皆为 void。换句话说我的意思是：

```
class B {
public:
    void mf();
    ...
};

class D: public B { ... };
```

虽然我们对 B、D 和 mf 一无所知，但面对一个类型为 D 的对象 x：

```
D x;           //x 是一个类型为 D 的对象
```

如果以下行为：

```
B* pB = &x;           //获得一个指针指向 x
pB->mf();              //经由该指针调用 mf
```

异于以下行为：

```
D* pD = &x;           //获得一个指针指向 x
pD->mf();              //经由该指针调用 mf
```

你可能会相当惊讶。毕竟两者都通过对象 x 调用成员函数 mf。由于两者所调用的函数都相同，凭借的对象也相同，所以行为也应该相同，是吗？

是的，理应如此，但事实可能不是如此。更明确地说，如果 mf 是个 non-virtual 函数而 D 定义有自己的 mf 版本，那就不是如此：

```
class D: public B {
public:
    void mf();           //遮掩 (hides) 了 B::mf; 见条款 33
    ...
};

pB->mf();                //调用 B::mf
pD->mf();                //调用 D::mf
```

造成此一两面行为的原因是，non-virtual 函数如 B::mf 和 D::mf 都是静态绑定 (statically bound, 见条款 37)。这意思是，由于 pB 被声明为一个 pointer-to-B，通